

1. Stored Procedure

A stored procedure is a reusable block of SQL/PLpgSQL code that performs one or more operations and can contain transaction control, loops, and conditional logic.

Example1:

```
CREATE OR REPLACE PROCEDURE insert_user(  
    p_user_id INT,  
    p_user_name VARCHAR,  
    p_email VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO users (user_id,user_name, department)  
    VALUES (p_user_id,p_user_name, p_email);  
  
    RAISE NOTICE 'User inserted successfully';  
END;  
$$;
```

Example2:

```
CREATE OR REPLACE PROCEDURE update_salary(  
    p_user_id INT,  
    p_salary NUMERIC
```

```
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE salary  
    SET salary_amount = p_salary  
    WHERE user_id = p_user_id;  
END;  
$$;
```

Execute:

```
CALL update_salary(1, 75000);
```

```
CREATE OR REPLACE PROCEDURE insert_user(  
    p_user_id INT,  
    p_user_name VARCHAR,  
    p_email VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF EXISTS (  
        SELECT 1
```

```
FROM users
WHERE user_id = p_user_id
) THEN
RAISE EXCEPTION 'User ID % already exists', p_user_id;
END IF;
```

```
INSERT INTO users(user_id, user_name, email)
VALUES (p_user_id, p_user_name, p_email);
```

```
RAISE NOTICE 'User inserted successfully';
END;
$$;
```

Use Case: Batch updates, ETL jobs, business processes.

2. Trigger

A trigger is code that automatically executes when an event occurs on a table (`INSERT`, `UPDATE`, `DELETE`).

Example Trigger Function:

```
CREATE OR REPLACE FUNCTION log_salary_change()
```

```
RETURNS TRIGGER
AS $$
BEGIN
    RAISE NOTICE 'Salary updated';
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Attach Trigger:

```
CREATE TRIGGER salary_update_trigger
AFTER UPDATE
ON salary
FOR EACH ROW
EXECUTE FUNCTION log_salary_change();
```

Use Case: Auditing, validation, automatic logging.

```
CREATE OR REPLACE FUNCTION log_employee_insert()
RETURNS TRIGGER
AS $$
BEGIN
    INSERT INTO employee_audit(message)
    VALUES ('Employee Added');
```

```
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
---
```

```
CREATE TRIGGER trg_employee_insert  
AFTER INSERT  
ON employees  
FOR EACH ROW  
EXECUTE FUNCTION log_employee_insert();
```

Types of Triggers in PostgreSQL

Triggers can be classified by **event** and **timing**.

Timing	Event
BEFORE INSERT	INSERT
AFTER INSERT	INSERT
BEFORE UPDATE	UPDATE
AFTER UPDATE	UPDATE
BEFORE DELETE	DELETE
AFTER DELETE	DELETE
AFTER TRUNCATE	TRUNCATE

3. User-Defined Function (UDF)

A custom function created by the developer.

Example:


```
CREATE OR REPLACE FUNCTION annual_salary(  
    monthly_salary NUMERIC  
)  
RETURNS NUMERIC  
AS $$  
BEGIN  
    RETURN monthly_salary * 12;  
END;  
$$ LANGUAGE plpgsql;
```

Usage:

```
SELECT annual_salary(50000);
```

Use Case: Reusable calculations and business rules.

Function vs Stored Procedure in PostgreSQL

Feature	Function	Stored Procedure
Purpose	Returns a value/result	Performs an action/process
Return Value	Mandatory (RETURNS)	Optional (no return required)
Call Syntax	SELECT function_name()	CALL procedure_name()
Can be used in SQL queries?	Yes	No
Transaction Control (COMMIT , ROLLBACK)	Not allowed	Allowed
Typical Use	Calculations, transformations	Insert, update, delete, business workflows



To get the newly inserted `user_id`, use `NEW.user_id`.

1. Clustered Index

A clustered index determines the **physical order of data** in a table.

PostgreSQL does **not have a true clustered index type** like SQL Server, but it supports table clustering.

1. Clustered Index

A clustered index determines the physical order of rows in a table.

In PostgreSQL:

- Create a normal index.
- Use **CLUSTER** to physically reorder the table based on that index.
- The ordering is not maintained automatically after inserts/updates.

Example

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    salary NUMERIC  
);
```

```
CREATE INDEX idx_salary  
ON employees(salary);
```

```
CLUSTER employees USING idx_salary;
```

After clustering, rows are physically stored by salary.

Query

```
SELECT *  
FROM employees  
WHERE salary BETWEEN 50000 AND 70000;
```

This can be faster because matching rows are stored close together.

2. Non-Clustered Index

Most PostgreSQL indexes are non-clustered.

- Separate structure from table data.
- Stores indexed column values and pointers to rows.
- Table data order remains unchanged.

Example

```
CREATE INDEX idx_emp_name  
ON employees(name);
```

Query

```
SELECT *  
FROM employees  
WHERE name = 'John';
```

PostgreSQL uses `idx_emp_name` to locate matching rows quickly.

Use Case: Fast searching without changing table storage order.

3. Composite Index

A composite index contains multiple columns.

Example

```
CREATE INDEX idx_dept_salary
ON employees(department_id, salary);
```

Table Data

depart ment_i d	sal ary
10	5000
10	6000

70
20 00
0

Queries Using Index

```
SELECT *  
FROM employees  
WHERE department_id = 10;
```

```
SELECT *  
FROM employees  
WHERE department_id = 10  
    AND salary > 50000;
```

1. Range Partitioning

Data is divided based on a range of values.

Example: Transactions by Year

```
CREATE TABLE transactions (  
    txn_id BIGINT,  
    txn_date DATE,  
    amount NUMERIC
```

```
) PARTITION BY RANGE (txn_date);
```

Partitions:

```
CREATE TABLE transactions_2025  
PARTITION OF transactions  
FOR VALUES FROM ('2025-01-01')  
TO ('2026-01-01');
```

```
CREATE TABLE transactions_2026  
PARTITION OF transactions  
FOR VALUES FROM ('2026-01-01')  
TO ('2027-01-01');
```

Insert:

```
INSERT INTO transactions  
VALUES (1, '2026-03-10', 1000);
```

Automatically goes to transactions_2026.

Query

```
SELECT *  
FROM transactions
```

```
WHERE txn_date BETWEEN '2026-01-01'  
AND '2026-12-31';
```

PostgreSQL reads only the 2026 partition.

Best for: Dates, timestamps, salary ranges, age ranges.

2. List Partitioning

Data is divided based on specific values.

Example: Customers by Country

```
CREATE TABLE customers (  
    customer_id INT,  
    country VARCHAR(50)  
) PARTITION BY LIST (country);
```

Partitions:

```
CREATE TABLE customers_india  
PARTITION OF customers  
FOR VALUES IN ('India');
```

```
CREATE TABLE customers_usa  
PARTITION OF customers
```



```
FOR VALUES IN ( 'USA' );
```

Insert:

```
INSERT INTO customers  
VALUES (1, 'India');
```

Goes to `customers_india`.

Query

```
SELECT *  
FROM customers  
WHERE country = 'India';
```

Only the India partition is scanned.

Best for: Country, State, Region, Status.

3. Hash Partitioning

Data is distributed using a hash function.

Example

```
CREATE TABLE orders (
```

```
    order_id BIGINT,  
    customer_id INT  
  ) PARTITION BY HASH (customer_id);
```

Partitions:

```
CREATE TABLE orders_p0  
PARTITION OF orders  
FOR VALUES WITH (MODULUS 4, REMAINDER 0);
```

```
CREATE TABLE orders_p1  
PARTITION OF orders  
FOR VALUES WITH (MODULUS 4, REMAINDER 1);
```

```
CREATE TABLE orders_p2  
PARTITION OF orders  
FOR VALUES WITH (MODULUS 4, REMAINDER 2);
```

```
CREATE TABLE orders_p3  
PARTITION OF orders  
FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

PostgreSQL hashes `customer_id` and places rows in one of the 4 partitions.

Query


```
SELECT *  
FROM orders  
WHERE customer_id = 1001;
```

PostgreSQL knows exactly which partition contains the row.

Best for: Even distribution of very large tables.

7. COMMIT

Permanently saves transaction changes.

```
COMMIT;
```

Use Case: Confirm database changes.

8. ROLLBACK

Undoes changes in the current transaction.

```
ROLLBACK;
```

Use Case: Error recovery.

9. SAVEPOINT

Creates a checkpoint inside a transaction.

```
SAVEPOINT sp1;
```

Rollback to savepoint:

```
ROLLBACK TO SAVEPOINT sp1;
```

Use Case: Partial rollback.

10. Covering Index

An index containing all columns needed by a query.

Index:

```
(user_id, user_name, department)
```

The database can answer the query using only the index without reading the table.

Use Case: Faster query performance.

11. Filtered Index (Partial Index in PostgreSQL)

Index only a subset of rows.

```
CREATE INDEX idx_active_users  
ON users(user_id)  
WHERE status = 'ACTIVE';
```

Use Case: Smaller index, faster lookups.

12. Columnstore Index

Stores data by columns instead of rows.

Traditional:
Row1 -> A,B,C
Row2 -> D,E,F

Columnstore:
A,D
B,E
C,F
Use Case: Analytics, reporting, data warehouses.

Interview One-Liners

Topic	One-Line Explanation
Clustered Index	Defines physical storage order of rows.
Non-Clustered Index	Separate lookup structure pointing to rows.
Composite Index	Index on multiple columns.
Range Partitioning	Splits data by value ranges.
List Partitioning	Splits data by predefined values.
Hash Partitioning	Splits data using hash distribution.
COMMIT	Permanently saves transaction changes.
ROLLBACK	Reverts transaction changes.
SAVEPOINT	Creates a rollback checkpoint.

Covering Index	Contains all columns required by a query.
Filtered Index	Indexes only selected rows.
Columnstore Index	Stores data column-wise for analytics.

PostgreSQL Note

-  Composite Index
-  Range Partitioning
-  List Partitioning
-  Hash Partitioning
-  SAVEPOINT
-  Partial (Filtered) Indexes

PostgreSQL does **not** have true Clustered Indexes like SQL Server. It has the `CLUSTER` command, which physically reorders a table once, but the order is not automatically maintained. Similarly, PostgreSQL does not have a native Columnstore Index like SQL Server; columnar storage is typically provided through extensions or specialized systems.

1. Covering Index

A covering index contains **all columns required by a query**, so PostgreSQL can answer the query using only the index without reading the table.

Example Query

```
SELECT user_name, department
FROM users
WHERE user_id = 1;
```

Index

```
CREATE INDEX idx_users_covering
ON users(user_id)
INCLUDE (user_name, department);
```

One-Liner:

A covering index contains all columns needed by a query, reducing table access.

2. Filtered Index (Partial Index in PostgreSQL)

An index created only for rows matching a condition.

Example

```
CREATE INDEX idx_active_users
```

```
ON users(user_id)
WHERE status = 'ACTIVE';
```

Only active users are indexed.

One-Liner:

A filtered index stores only selected rows, making the index smaller and faster.

3. Columnstore Index

Stores data **column by column** instead of row by row.

Traditional row storage:

```
1, John, IT
2, Mary, HR
3, David, Finance
```

Column storage:

```
ID:          1, 2, 3
Name:        John, Mary, David
Department:  IT, HR, Finance
```

Why?

Analytics queries often read only a few columns:

```
SELECT AVG(salary_amount)
FROM salary;
```

Only the salary_amount column needs to be read.

One-Liner:

A columnstore index stores data by columns for fast analytical and reporting queries.

Interview Memory

Index Type	Purpose
Covering Index	Avoid table lookup
Filtered Index	Index only important rows
Columnstore Index	Speed up analytics/reporting queries

Easy Story

- **Covering Index** → "Everything I need is already in the index."
- **Filtered Index** → "Index only active customers."
- **Columnstore Index** → "Read only the columns needed for reports."